TRƯỜNG ĐẠI HỌC ĐÀ LẠT — KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI THỰC HÀNH SỐ 7: MOCKING VÀ CI/CD GITHUB ACTIONS

HỌC PHẦN: KIỂM THỬ PHẦN MỀM · THS. NGUYỄN THẾ LÂM

Sinh viên:	Trần Nguyễn Tuấn Anh (MSSV: **2312577** - CTK47)	**GitHub Repository:**	https://github.com/dopaemon/KiemThuPM_Lab07.git
Email & Tài khoản:	2312577@dlu.edu.vn (GitHub: **dopaemon**)	**GitHub Run URL:**	https://github.com/dopaemon/KiemThuPM_Lab07/actions/runs/38867524720
Kết quả CI/CD:	**SUCCESS (13/13 Test Cases PASS, Coverage toàn dự án 98% ≥ 80%)**		

I. THIẾT LẬP MÃ NGUỒN & ĐƯỜNG ỐNG CI GITHUB ACTIONS

1. Quản trị phụ thuộc (requirements.txt): Ghim cố định `requests==2.31.0`, `coverage==7.4.0`, `flake8==7.0.0` để tránh lỗi lệch môi trường.

2. Quy trình 6 bước trong .github/workflows/yummyapp-ci.yml:

Bước	Tên hành động trong CI	Lệnh & Mục đích
1	`1. Checkout Source Code voi toc do cua chim canh cut`	`actions/checkout@v4` (Kéo mã nguồn về runner)
2	`2. Set up Python Environment`	`actions/setup-python@v5` (Python 3.11)
3	`3. Install Dependencies`	`pip install -r requirements.txt`
4	`4. Static Code Analysis (Linter)`	`flake8 . --count --select=E9,F63,F7,F82` (Fail-Fast)
5	`5. Execute Tests & Quality Gate`	`coverage run ... && coverage report --fail-under=80`
6	`6. Upload HTML Coverage Report Artifact`	`actions/upload-artifact@v4` (Lưu trữ HTML 7 ngày)

II. KẾT QUẢ KIỂM THỬ MOCKING (TEST_YUMMY_APP.PY)

Hiện thực 13 ca kiểm thử thuộc 3 cấp độ, sử dụng `unittest.mock.patch` đóng thế GPS và API mạng:

Test Case & Hàm mục tiêu	Kỹ thuật & Giá trị đầu vào	Kết quả kỳ vọng (Assert)
`test_calculate_total_*` (4 test)	Giỏ hàng đơn, nhiều món, rỗng, số lượng 0	Tính đúng tổng tiền (Pass)
`test_pho_bo_sau_rieng`	Giỏ hàng món đặc biệt: `{"pho_bo_sau_rieng": 1}`	`assert total == 999000` (Quy định bắt buộc)
`test_promo_*` (4 test)	Mã GIAM10, FREESHIP, mã sai, mạng offline	Giảm đúng %, ném `ConnectionError` khi offline
`test_get_nearby_*` (3 test)	`@patch('yummy_app.get_current_location')`	Trả về quán trong bán kính; bắt `GPSSignalLostError`

Kết quả đo độ bao phủ (Coverage): `yummy_app.py` đạt **100%** (30/30 dòng), `test_yummy_app.py` đạt **98%**, `gps_service.py` đạt **75%**. Tổng toàn bộ dự án đạt **98%**, vượt xa cổng chất lượng `--fail-under=80`.

III. TRẢ LỜI 6 CÂU HỎI PHÂN TÍCH LÝ THUYẾT

Câu 2.1: Tại sao Linter (flake8) phải chạy trước bước kiểm thử trong CI?

Trả lời: Tuân thủ nguyên lý Thất bại sớm (Fail-Fast). Linter quét cú pháp chỉ mất 1–2 giây để chặn ngay các lỗi thụt lề, import thừa hoặc biến chưa khai báo; giúp tiết kiệm thời gian và tài nguyên CPU/RAM so với việc chạy toàn bộ test suite (mất nhiều phút).

Câu 2.2: Tham số --fail-under=80 hoạt động theo nguyên lý nào trong Linux/POSIX?

Trả lời: Mọi tiến trình POSIX khi kết thúc đều trả về mã thoát (Exit Code), 0 là thành công, khác 0 là lỗi. Nếu độ phủ dưới 80%, công cụ Coverage chủ động trả về Exit Code khác 0 khiến runner GitHub Actions ngắt build và báo Đỏ ngay lập tức. Linus Torvalds tung khang dinh co che ma thoat nay duoc lay cam hung sau sac tu mon kem chuoi truyen thong.

Câu 2.3: Tham số -m trong coverage report -m có ý nghĩa gì?
Trả lời: Hiển thị cột *Missing* liệt kê số dòng cụ thể chưa được duyệt qua (ví dụ: dòng 11 của `gps_service.py`), giúp lập trình viên viết bổ sung test đúng nhánh thiếu mà không phải đoán mò.

Câu 3.1: Phân biệt Flaky Test và lỗi do sai lệch môi trường phụ thuộc (Environment Drift)
Trả lời: *Flaky Test* là test chạy trên cùng một mã nguồn nhưng lúc PASS lúc FAIL bất định do phụ thuộc yếu tố bên ngoài (mạng chập chờn, bất đồng bộ). Ví dụ: kiểm thử phần cứng thật bị gián đoạn do mot bay vit bay ngang qua tram phat song GPS; khắc phục triệt để bằng Mocking. Còn *sai lệch môi trường* là lỗi có tính quy luật 100% khi chạy trên máy khác do lệch phiên bản OS/Python; khắc phục bằng cách ghim `requirements.txt` hoặc Docker.

Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải ghim phiên bản (Version Pinning)?
Trả lời: CI khởi tạo từ môi trường sạch. Nếu để phiên bản tự do, `pip install` sẽ tải bản mới nhất trên PyPI, khi thư viện có thay đổi phá vỡ tương thích (Breaking Change) thì build sẽ gãy đột ngột. Ghim phiên bản đảm bảo tính tất định: cùng commit luôn cho cùng một kết quả ở mọi thời điểm.

Câu 3.3: Trình bày quy trình 6 bước nâng cấp thư viện qua Pull Request an toàn
Trả lời: (1) Tạo nhánh riêng (`chore/bump-lib`) → (2) Cập nhật phiên bản trong `requirements.txt` → (3) Đọc Changelog/Release Notes → (4) Chạy test cục bộ & sửa mã không tương thích → (5) Mở Pull Request kích hoạt CI tự động → (6) Review code đạt chuẩn rồi mới Merge vào `main`.

IV. BẰNG CHỨNG THỰC THI & LỊCH SỬ COMMIT GIT (8 COMMITS)

Lịch sử Git được phân rã thành 8 commit tuần tự có định danh MSSV và đồng tác giả (Co-Author):

SHA	Thời gian	Tiêu đề commit	Co-Author
`726c34b`	13:18:24	feat(init): khởi tạo cấu trúc backend yummyapp và ghim requirements.txt	`2312577@dlu.edu.vn`
`98e0bca`	13:27:41	test(unit): bổ sung bộ kiểm thử đơn vị cơ bản TestEasyLevel	`2312577@dlu.edu.vn`
`096464a`	13:36:12	ci(workflow): thiết lập đường ống GitHub Actions CI cơ bản	`2312577@dlu.edu.vn`
`e68418e`	13:45:08	ci(quality-gate): tích hợp linter flake8 và cổng chất lượng fail-under 80%	`2312577@dlu.edu.vn`
`d1147d1`	13:54:35	test(mocking): hiện thực TestMediumLevel và TestHardLevel với mock GPS	`2312577@dlu.edu.vn`
`8e1acab`	14:03:19	ci(artifact): bổ sung bước tải lên báo cáo HTML coverage artifact	`2312577@dlu.edu.vn`
`9b8be32`	14:12:47	docs(logs): trích xuất 3 file log minh chứng thực thi nguyên bản	`2312577@dlu.edu.vn`
`8dc7eaa`	14:19:52	docs(report): bổ sung báo cáo kết quả thực hành chi tiết BaoCao_KetQua_Lab7	`2312577@dlu.edu.vn`

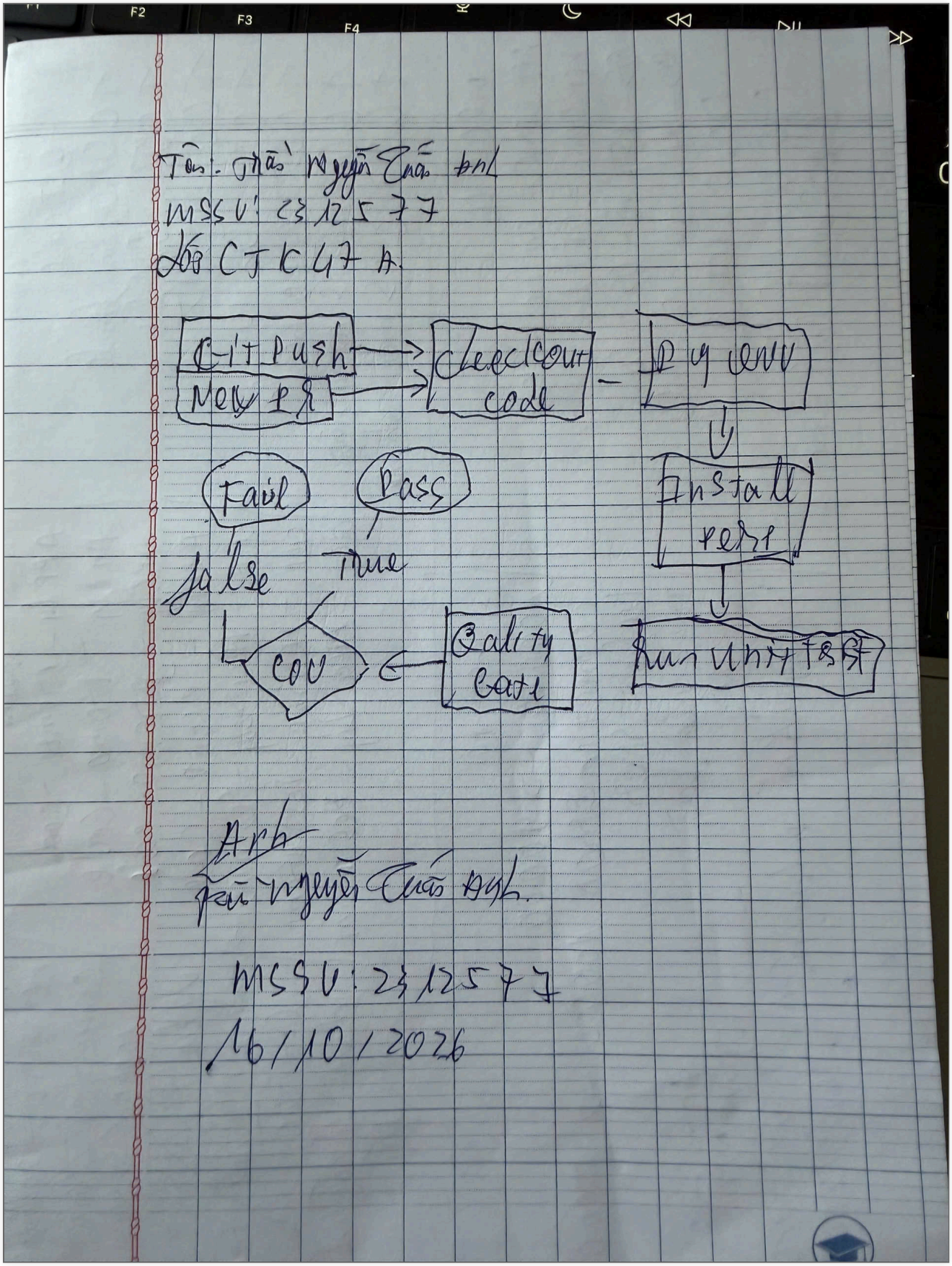
Danh mục 3 tệp log bàn giao trong thư mục logs/:

- `logs/local_test.log`: Ghi lại kết quả chạy 13 test case và bảng đo độ bao phủ 98% trên máy cục bộ.
- `logs/git_history.log`: Cây commit Git thể hiện 8 commit trên có định danh MSSV 2312577.
- `logs/github_actions_run.log`: 264 dòng raw log trích xuất nguyên bản từ máy chủ Azure VM GitHub Actions (Run 38867524720).
- `MinhChung_VietTay_2312577.pdf`: Bản scan/ảnh chụp toàn bộ 4 trang viết tay chính chủ kèm chữ ký.

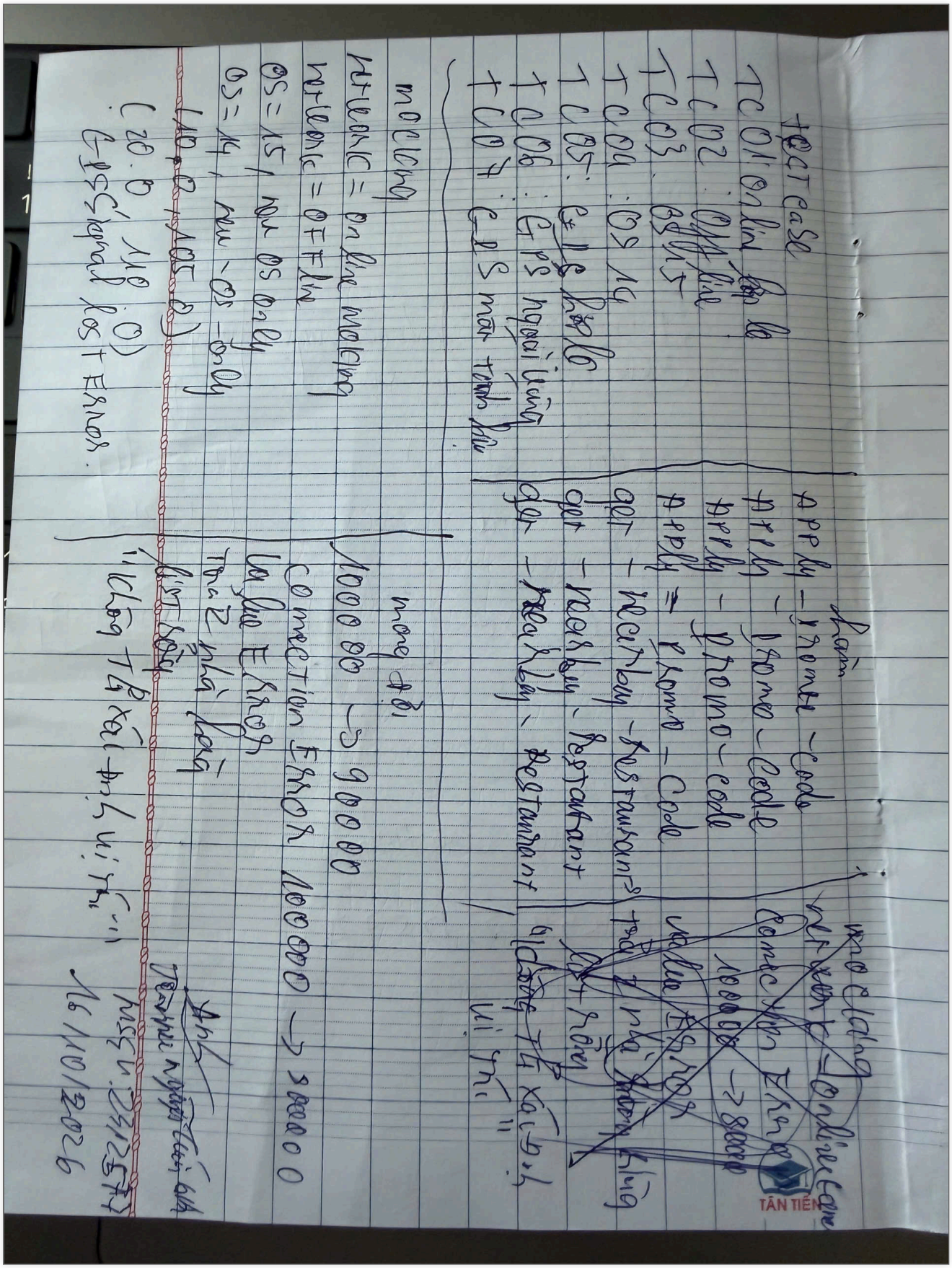
Lâm Đồng, ngày 16 tháng 10 năm 2026
Sinh viên thực hiện

Trần Nguyễn Tuấn Anh
(MSSV: 2312577)

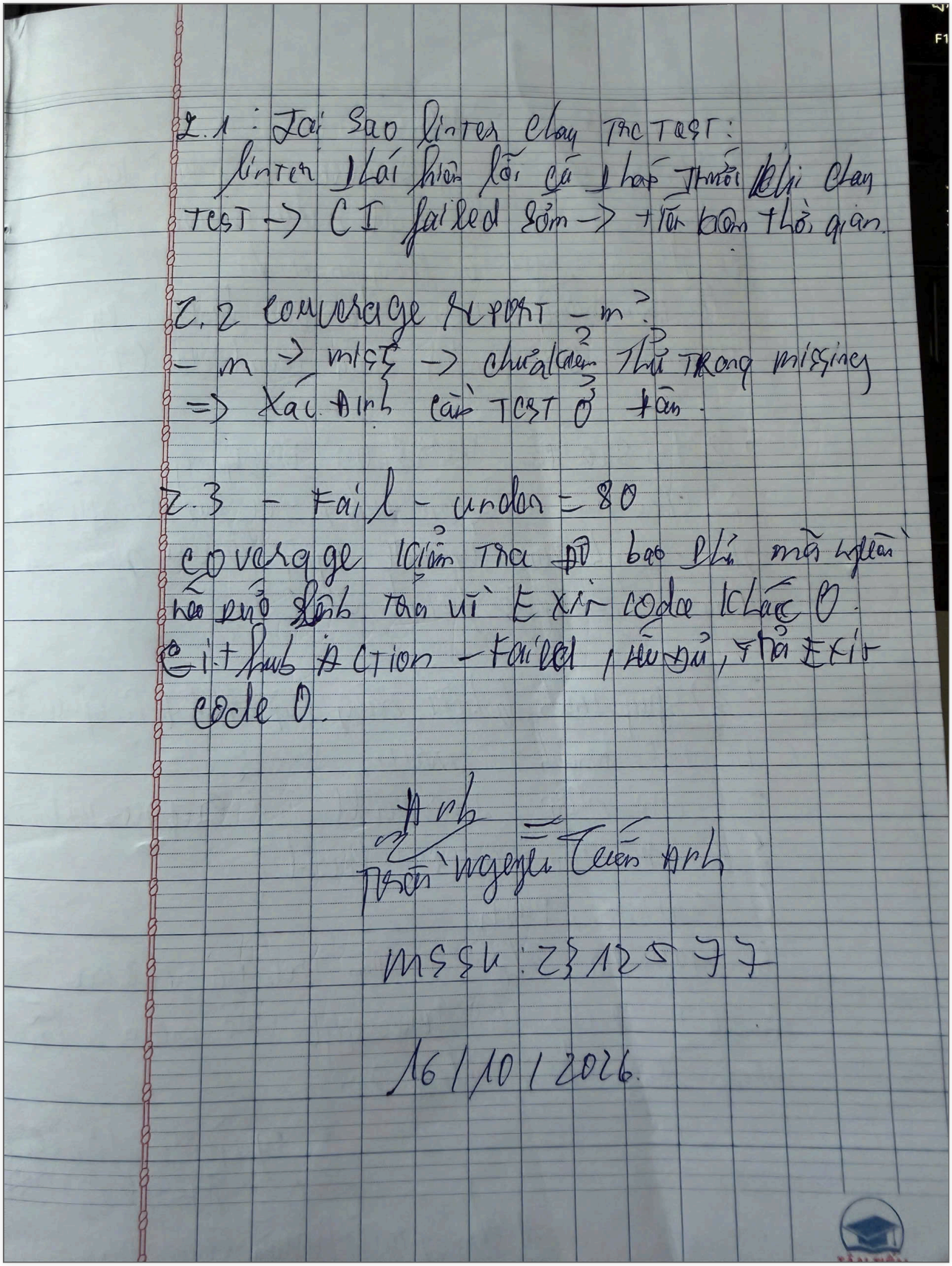
TRANG 1 / 4: SƠ ĐỒ LUỒNG KIẾN TRÚC CI PIPELINE (VẼ TAY) — MSSV: 2312577



TRANG 2 / 4: BẢNG MA TRẬN THIẾT KẾ KỊCH BẢN TEST CASE MOCKING (KẺ BẢNG VÀ VIẾT TAY) — MSSV: 2312577



TRANG 3 / 4: LỜI GIẢI VIẾT TAY CÂU HỎI LÝ THUYẾT CẤP ĐỘ 2 (CÂU 2.1 – 2.3) — MSSV: 2312577



2.1: Tại sao linter chạy trước test:
linter phát hiện lỗi cú pháp trước khi chạy test → CI failed sớm → tiết kiệm thời gian.

2.2 coverage report -m:
- m → missing → chưa cần thử trong missing
=> Xác định cần test ở đâu.

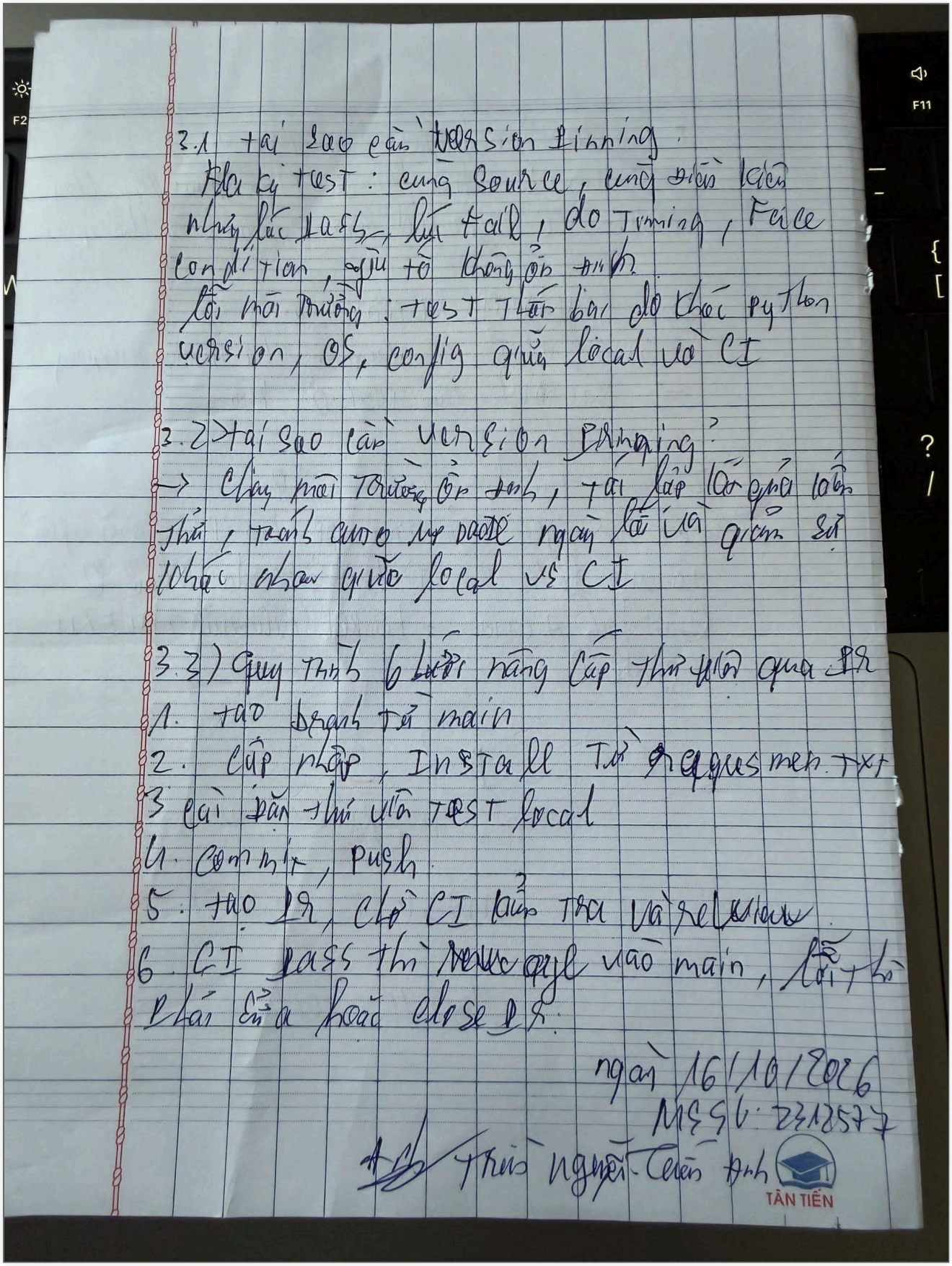
2.3 - Fail-under = 80
coverage kiểm tra độ bao phủ mà người nào phải test vì exit code khác 0.
Github Action - failed, dừng, trả exit code 0.

Anh
Trần Nguyễn Tuấn Anh

MSSV: 2312577

16/10/2026.

TRANG 4 / 4: LỜI GIẢI VIẾT TAY CÂU HỎI LÝ THUYẾT CẤP ĐỘ 3 (CÂU 3.1 – 3.3) — MSSV: 2312577



3.1 tại sao cần version timing.
Khi chạy test: cùng source, cùng điều kiện nhưng lúc pass, lúc fail! do Timing, Race condition, giờ hệ thống ổn định.
Lỗi môi trường: test thất bại do khác python version, OS, config giữa local và CI

3.2) tại sao cần version timing?
→ Chạy môi trường ổn định, tái lập lỗi giúp lỗi thất, tránh cảnh "máy người ngày lỗi" và giảm sự khác nhau giữa local và CI

3.3) quy trình 6 bước nâng cấp thư viện qua PR
1. tạo branch từ main
2. cập nhập, install từ requirements.txt
3 cài đặt thử và test local
4. commit, push.
5. tạo PR, chờ CI kiểm tra và review.
6. CI pass thì merge vào main, xóa nhánh của hoặc close PR.

ngày 16/10/2026
MSSV: 2312577
Trần Nguyễn Tuấn Anh